
PracticePro365 Discussion

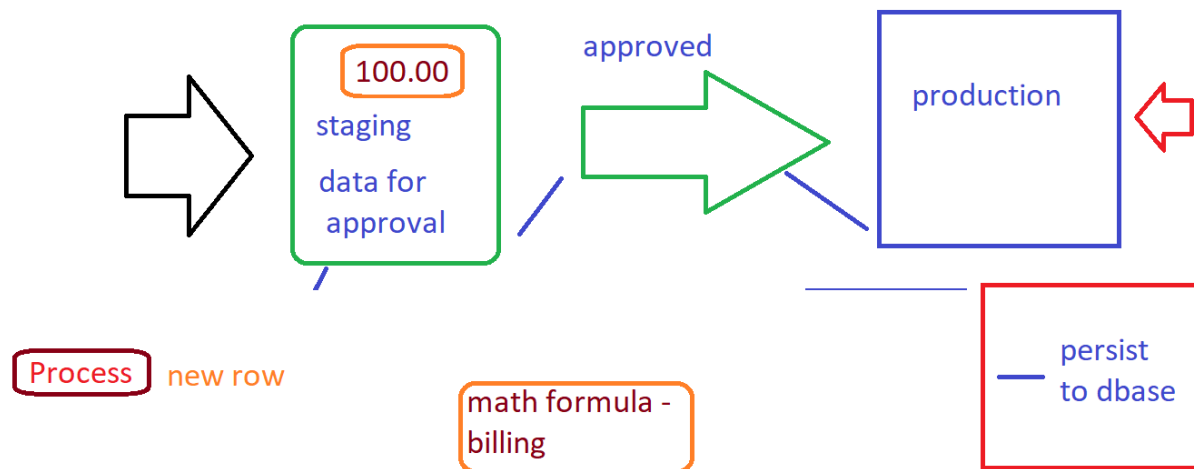
Contents

Possible sub flow changes:.....	2
Recommended Pattern	3
Phase 1: Create the record	3
Phase 2: Find the newly created record and update it	3
Option A (Best): Pass a Correlation Key	4
Option B: Created By + Timestamp	4
Option C: Trigger from Dataverse Instead	5
Option D: Use Process Completion Output	5
Architecture I Would Use	6
What's happening?	8
Best Practice #1: Put customer customizations in a separate solution	9
Vendor Solution	9
Customer Solution	9
The Form Overwrite Problem	9
Better Option: Create a Separate Form	10
Vendor	10
Client	10
Best Practice #2: Use Solution Layering Intentionally	10
Can you export the form before upgrades and re-import it?	11
Better Enterprise Approach	11
Option A (Preferred)	11
Option B	12
Option C (Most Mature)	12
What I would do	12

Possible sub flow changes:

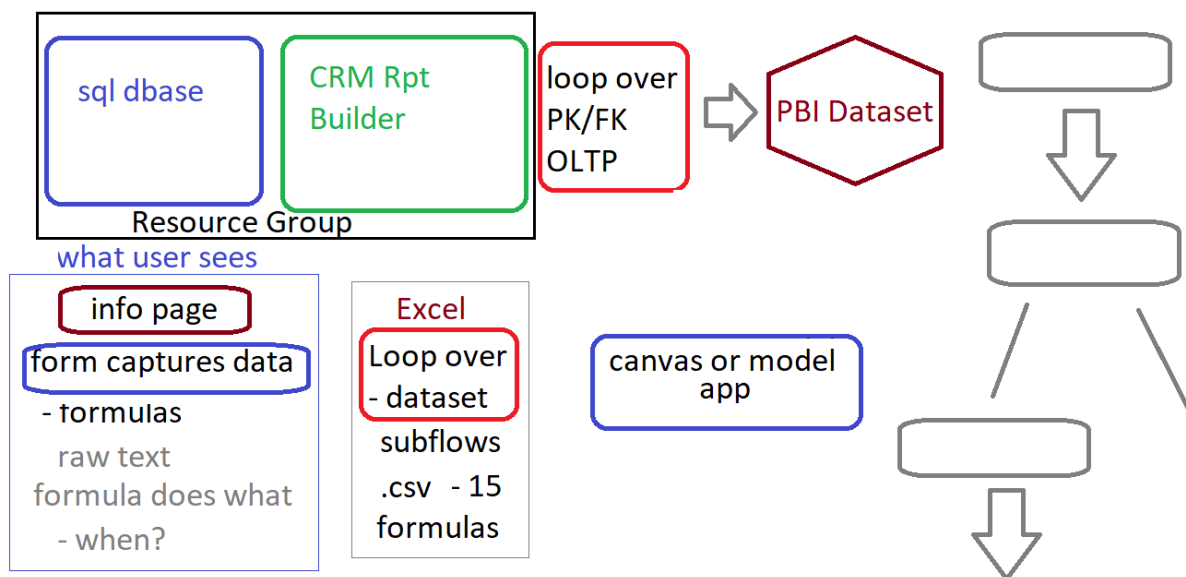
1. revise the loop
2. mock 2/3 collection of excel data
 - model driven app?
3. *Use AI Build to find field to insert to 365 Dataverse
4. Work Order app

[Automate using UI elements - Power Automate | Microsoft Learn](#)



- the PracticePro function to add the specific record across the tables
- Then insert the value to the new field

we do not know what to add to Dataverse unless it is created by the PracticePro function first.



[Add logic to commands with Power Fx - Training | Microsoft Learn](#)

Based on your description, it sounds like the core challenge is:

1. **PracticePro 365 creates records across multiple Dataverse tables.**
2. The record ID (or related IDs) are not known until the PracticePro function completes.
3. Due to governance restrictions, Power Automate cannot directly discover/query Dataverse metadata or tables.
4. A macro/UI automation is currently being used to imitate user actions to create the record.

Recommended Pattern

Instead of trying to populate the field during creation, split the process into **two phases**:

Phase 1: Create the record

Trigger your macro or PracticePro function first.

```
1 Power Automate
2   ↓
3 Desktop Flow / Macro
4   ↓
5 PracticePro 365 Function
6   ↓
7 Creates record(s) in Dataverse
```

Phase 2: Find the newly created record and update it

After PracticePro finishes:

```
1 Wait
2   ↓
3 Look up newly created record
4   ↓
5 Get Record ID
6   ↓
7 Update target field
```

The challenge becomes **how to identify the newly created record**.

Option A (Best): Pass a Correlation Key

If PracticePro allows any custom field value to be populated during creation, generate a unique value first.

Example:

```
1 PP-20260820-123456
```

Store it in Power Automate.

Then:

1. Generate GUID

```
1 guid()
```

1. Pass GUID into PracticePro creation process.
2. PracticePro creates records.
3. Power Automate searches Dataverse for that GUID.
4. Once found:
 - Get Record ID
 - Update the additional field(s)

This is the most reliable enterprise pattern.

Option B: Created By + Timestamp

If no custom value can be passed:

1. Capture current UTC timestamp.

```
1 utcNow()
```

1. Run PracticePro creation.
2. Query records:
 - Created By = automation account
 - Created On > timestamp captured earlier
3. Sort descending by Created On.
4. Use newest record.

Example logic:

```
1 Start Time  
2 ↓
```

```
3 Create Record via PracticePro
4   ↓
5 List Rows
6   Filter:
7     CreatedOn gt StartTime
8   Order By:
9     CreatedOn desc
10  Top Count:
11    1
```

This works well when transaction volume is low.

Option C: Trigger from Dataverse Instead

If governance allows Dataverse triggers but not discovery:

```
1 When a row is added
2   ↓
3 Check if created by PracticePro
4   ↓
5 Update required field
```

In this model:

- PracticePro creates the record.
- Dataverse triggers the flow.
- Flow updates the new field.

This removes the need to find the record afterward.

Option D: Use Process Completion Output

If the macro/Desktop Flow can read the UI after creation:

```
1 Create Matter
2   ↓
3 Capture Matter Number
4   ↓
5 Return Matter Number to Cloud Flow
6   ↓
```

```
7  Dataverse Lookup by Matter Number
8      ↓
9  Update field
```

Many organizations use Power Automate Desktop this way:

- Click Create
- Read confirmation screen
- Extract generated ID
- Return value to cloud flow

If PracticePro displays the generated Matter ID, Client ID, File Number, etc., this may be your cleanest solution.

Architecture I Would Use

```
1  Cloud Flow
2      ↓
3  Generate Correlation GUID
4      ↓
5  Run Desktop Flow
6      ↓
7  PracticePro Create Record
8      ↓
9  Store GUID/Reference
10     ↓
11  Return generated Matter Number
12     ↓
13  Lookup Dataverse Record
14     ↓
15  Update New Field
```

This avoids relying on "latest created record" logic and is much more resilient.

The key question technically is not "how do we update the field?" but rather **what unique value can be carried through the PracticePro creation process that lets Power Automate identify the newly created Dataverse row afterward?** Once that identifier exists, the update step becomes straightforward.

PracticePro 365 Integration Options

PracticePro 365 is a cloud-based accounting practice management platform for CPA and tax firms, built natively on Microsoft's Power Platform and Dataverse. It supports deep integration with Microsoft 365 tools like Outlook, Teams, SharePoint, and Power BI, and offers automation via **Microsoft Power Automate** [PracticePro 365](#).

API and SDK Options

While the provided results do not list a public REST API or SDK documentation, PracticePro 365's Microsoft-native architecture suggests it is designed to be exposed through:

- **RESTful endpoints** for programmatic access to data and workflows (common in Power Platform apps).
- **Power Automate connectors** for event-driven workflows and data movement [PracticePro 365](#).
- **Microsoft Graph API** for identity, SSO, and some data sync scenarios.
- **Custom SDKs** (likely in .NET or similar) for developers building firm-specific integrations.

Because it's deployed in your own Microsoft tenant, integration is typically done via:

- **Power Platform connectors** (if available in the marketplace).
- **Custom connector development** using Microsoft's documentation for the target service.
- **Direct REST API calls** to the PracticePro 365 backend (if exposed).

SOAP Service

There is no explicit mention of a SOAP service in the provided results. Given that PracticePro 365 is a modern Microsoft Power Platform app, it is more likely to be accessed via **REST APIs** rather than SOAP. If you need SOAP, you would need to confirm with PracticePro 365 support whether they expose a SOAP endpoint or if you can use a middleware layer to translate between REST and SOAP.

GitHub Automation Scripts

The [practicepro365/scripts](#) repository contains public automation scripts, which may be useful for:

- Automating data exports or imports.
- Integrating with Azure Synapse or other data platforms.

- Running scheduled tasks in your firm's environment [Github](#).

These scripts can serve as a starting point for developers familiar with Azure and Power Platform.

Next Steps

1. **Contact PracticePro 365 Support** — Ask about official REST API documentation, SDK availability, and whether SOAP is supported.
2. **Check the Microsoft Power Platform Marketplace** — Look for a “PracticePro 365” connector.
3. **Review GitHub scripts** — Adapt them for your integration needs.
4. **Plan for Microsoft Identity** — Use Azure AD SSO and Microsoft Graph for secure access.

Tip: If you're building a custom integration, start with a REST API approach, as it's more aligned with the platform's architecture, and consider Power Automate for workflow automation.

If you'd like, you can request a demo from PracticePro 365 to discuss your integration requirements and get the most up-to-date API/SDK details.

Updates to app & Custom Controls

Hey Co - When an app like PracticePro365 is updated & published from the dev team

- a custom page with form fields in a model driven app gets an additional field added by client after updates from dev.

- how can custom field and form changes made in classic dynamics by the client not get overwritten on a new published form. (the field remains but the form gets overwritten.

can I export the custom form & re import after an update or package in a solution to manage changes better?

Yes. What you're running into is a very common Dynamics 365 / Power Apps ALM issue.

What's happening?

When the dev team updates and imports a managed solution (or publishes customizations), the **form definition** included in that solution replaces the existing form definition in the target environment.

-  The custom field itself often survives because it exists as a separate metadata component.
-  The form layout gets overwritten because the form is being delivered from the development solution.

So after an update:

1	Field: ClientCustomField	-> Still exists
2	Form: Main Form	-> Replaced by vendor/dev version
3	Result: Field disappears from the UI	

Best Practice #1: Put customer customizations in a separate solution

The recommended approach is:

Vendor Solution

1	PracticePro365 (Managed)
---	--------------------------

Customer Solution

1	PracticePro365 - Client Customizations (Unmanaged)
---	--

Store:

- Custom fields
- Form modifications
- Views
- Business rules
- Model-driven app customizations

in the customer solution.

This creates a cleaner ALM boundary.

However, there is an important caveat...

The Form Overwrite Problem

Even if the client changes the form through a separate unmanaged solution:

- If the vendor imports a newer version of the same form

- The new form XML becomes the active layer

The customer changes may not appear.

This is a **solution layering** issue.

Better Option: Create a Separate Form

Instead of modifying the vendor's main form:

Vendor

1 PracticePro Main Form

Client

Create

1 PracticePro Main Form - Client

Add:

- Client fields
- Client tabs
- Client sections

Then assign appropriate security roles.

Benefits:

- Vendor updates won't overwrite it.
- Client controls their own form lifecycle.
- Easier upgrades.

This is often the safest approach for ISV solutions.

Best Practice #2: Use Solution Layering Intentionally

Check:

1 Power Apps
2 → Solution Layers
3 → Form

You can inspect:

- Managed layer (vendor)
- Unmanaged layer (client)

If the vendor update keeps replacing the form, you can see exactly which layer is winning.

Can you export the form before upgrades and re-import it?

Technically yes.

Process:

1. Create a customer solution.
2. Add the form component.
3. Export the solution.
4. Import vendor upgrade.
5. Re-import customer solution.

But this becomes:

- Error-prone
- Time consuming
- Difficult to maintain

I generally wouldn't recommend this as the primary strategy.

Think of it as a recovery mechanism rather than an ALM strategy.

Better Enterprise Approach

For PracticePro365, I'd recommend:

Option A (Preferred)

Create:

1 Client Extension Solution

containing:

- Custom tables
- Custom columns
- Custom forms
- Custom views

and use a dedicated Client form.

Option B

If users must stay on the vendor form:

- Keep all client customizations in a separate unmanaged solution.
- Document the changes.
- After every vendor release, validate that the form layer still contains the custom controls.

Option C (Most Mature)

Have the vendor provide an **extension solution layer** where customer customizations are expected to live.

Many ISVs build their app specifically for this pattern.

What I would do

If I were managing PracticePro365:

1. Keep the vendor solution managed.
2. Create a separate "PracticePro365 Client Extensions" solution.
3. Create a new Main Form specifically for the client.
4. Add all client fields/tabs there.
5. Leave the vendor form untouched.
6. Include the custom form in source control and deployment pipelines.

That approach survives upgrades far better than trying to continuously merge form changes after every release.